

## Лабораторная работа №10

### AVL-дерево. Префиксное дерево (Trie)

Цель работы:

1. Реализовать самобалансирующееся AVL-дерево с операциями вставки, удаления и поиска
2. Освоить механизмы балансировки: малые и большие повороты (правый, левый, правый-левый, левый-правый)
3. Изучить структуру данных «префиксное дерево» (Trie) для эффективного хранения и поиска строк
4. Реализовать операции вставки, поиска и удаления в Trie
5. Сравнить производительность AVL-дерева, BST и Trie для различных типов данных

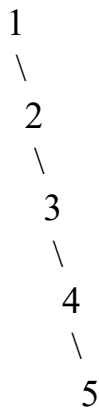
### Теоретическое обоснование

#### 1. Проблема вырождения BST

Обычное бинарное дерево поиска (BST) может вырождаться в связный список при вставке элементов в отсортированном порядке.

Вырожденное дерево (худший случай):

Вставка: 1, 2, 3, 4, 5



Поиск 5 требует 5 шагов  $\rightarrow O(n)$

#### 2. AVL-дерево

AVL-дерево - это самобалансирующееся бинарное дерево поиска, в котором для каждого узла выполняется условие:

$|\text{высота(левое поддерев)} - \text{высота(правое поддерев)}| \leq 1$

Фактор баланса:

```
int balanceFactor(Node* node) {
```

```

    return height(node->left) - height(node->right);
}

```

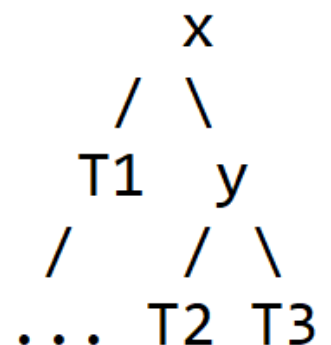
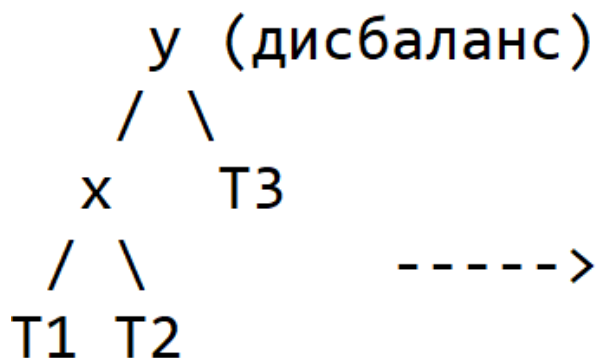
| Фактор баланса | Состояние             | Действие       |
|----------------|-----------------------|----------------|
| -1, 0, 1       | Норма                 | Ничего         |
| 2              | Левое поддерево выше  | Правый поворот |
| -2             | Правое поддерево выше | Левый поворот  |

### 3. Повороты

Правый поворот (Right Rotation)

Применяется при: Фактор баланса  $> 1$  и левый потомок имеет фактор баланса  $\geq 0$

Визуализация:



```

Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    updateHeight(y);
    updateHeight(x);

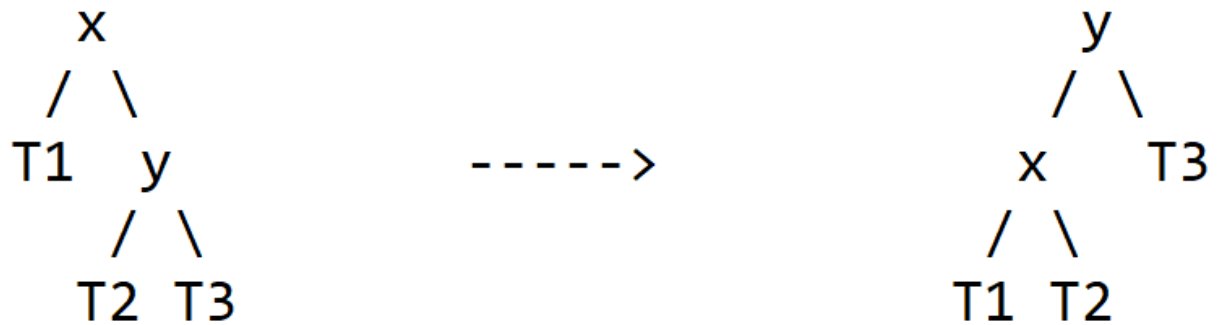
    return x;
}

```

### Левый поворот (Left Rotation)

Применяется при: Фактор баланса  $< -1$  и правый потомок имеет фактор баланса  $\leq 0$

Визуализация:



```
Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* T2 = y->left;

    y->left = x;
    x->right = T2;

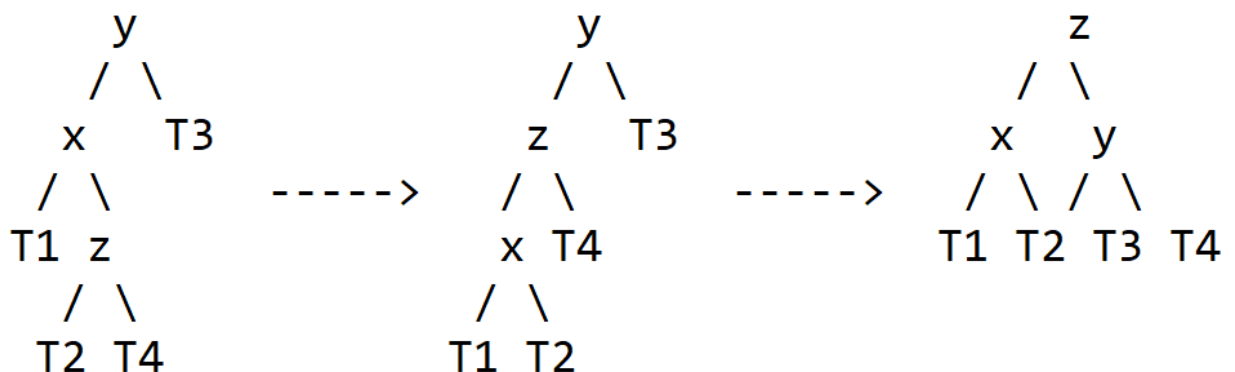
    updateHeight(x);
    updateHeight(y);

    return y;
}
```

### Левый-правый поворот (Left-Right Rotation)

Применяется при: Фактор баланса  $> 1$  и левый потомок имеет фактор баланса  $< 0$

Визуализация:

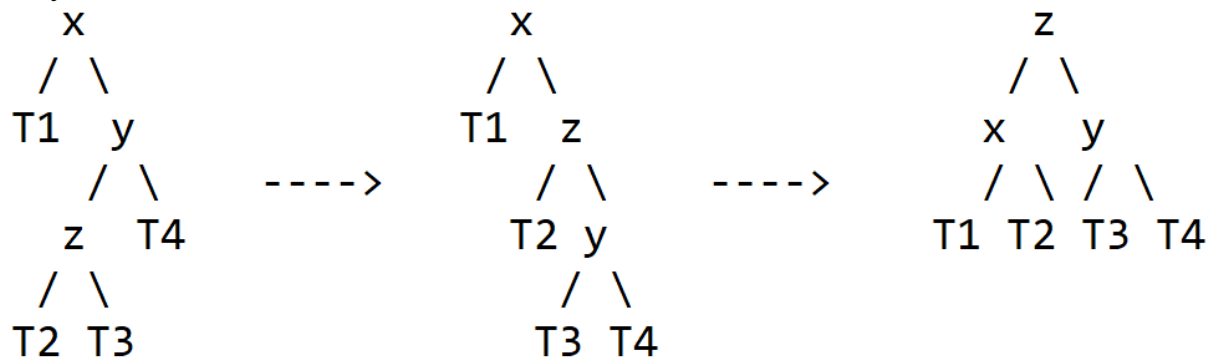


```
Node* leftRightRotate(Node* y) {
    y->left = leftRotate(y->left);
    return rightRotate(y);
}
```

Правый-левый поворот (Right-Left Rotation)

Применяется при: Фактор баланса  $< -1$  и правый потомок имеет фактор баланса  $> 0$

Визуализация:

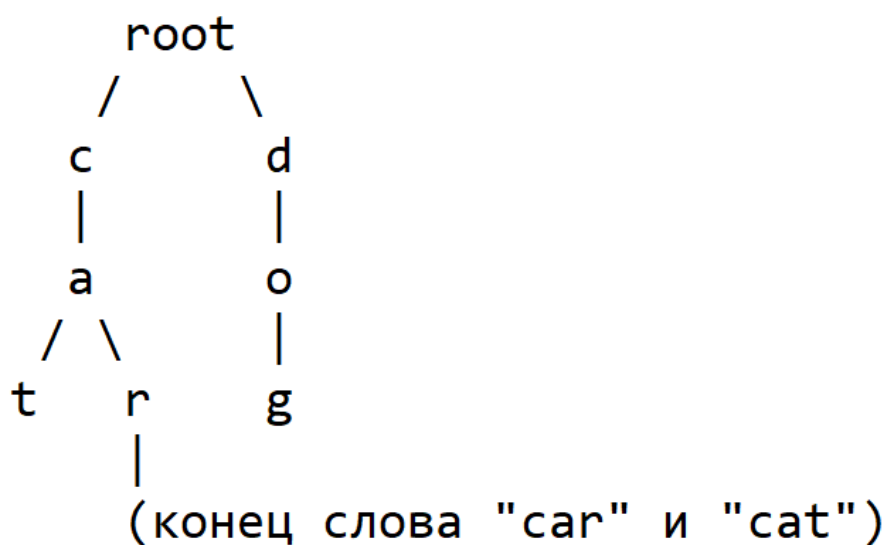


```
Node* rightLeftRotate(Node* x) {
    x->right = rightRotate(x->right);
    return leftRotate(x);
}
```

#### 4. Префиксное дерево (Trie)

Trie (префиксное дерево) - это структура данных для хранения строк, позволяющая быстро выполнять операции поиска, вставки и удаления.

Визуализация Trie для слов: "cat", "car", "dog"



Структура узла Trie:

```
struct TrieNode {
    TrieNode* children[26]; // для букв a-z
    bool isEndOfWord;       // маркер конца слова

    TrieNode() : isEndOfWord(false) {
        for (int i = 0; i < 26; i++) {
            children[i] = nullptr;
        }
    }
};
```

## 5. Операции с Trie

Вставка

```
void insert(const string& word) {
    TrieNode* current = root;
    for (char ch : word) {
        int index = ch - 'a';
        if (current->children[index] == nullptr) {
            current->children[index] = new TrieNode();
        }
        current = current->children[index];
    }
    current->isEndOfWord = true;
}
```

Поиск

```
bool search(const string& word) {
    TrieNode* current = root;
    for (char ch : word) {
        int index = ch - 'a';
        if (current->children[index] == nullptr) return false;
        current = current->children[index];
    }
    return current->isEndOfWord;
}
```

Поиск по префиксу

```
bool startsWith(const string& prefix) {
    TrieNode* current = root;
```

```

for (char ch : prefix) {
    int index = ch - 'a';
    if (current->children[index] == nullptr) return false;
    current = current->children[index];
}
return true; // есть хотя бы одно слово с данным префиксом
}

```

## 6. Сравнение структур данных

| Характеристика     | BST                   | AVL         | Trie                                    |
|--------------------|-----------------------|-------------|-----------------------------------------|
| Поиск (средний)    | $O(\log n)$           | $O(\log n)$ | $O(L)$<br>длина строки                  |
| Вставка (средний)  | $O(\log n)$           | $O(\log n)$ | $O(L)$                                  |
| Удаление (средний) | $O(\log n)$           | $O(\log n)$ | $O(L)$                                  |
| Худший случай      | $O(n)$                | $O(\log n)$ | $O(L)$                                  |
| Память             | $O(n)$                | $O(n)$      | $O(\text{алфавит} \times \text{длина})$ |
| Строковые операции | Медленные (сравнение) | Медленные   | Быстрые                                 |
| Автодополнение     | Нет                   | Нет         | Да                                      |

## Примеры выполнения практической части

Пример 1. AVL-дерево - вставка и повороты

Задача: Реализовать AVL-дерево с операциями вставки и визуализацией балансировки.

```

#include <iostream>
#include <algorithm>

using namespace std;

struct AVLNode {
    int key;

```

```
AVLNode* left;
AVLNode* right;
int height;
```

```
AVLNode(int k) : key(k), left(nullptr), right(nullptr), height(1) {}
};
```

```
class AVLTree {
private:
```

```
    AVLNode* root;
```

```
    int height(AVLNode* node) {
        return node ? node->height : 0;
    }
```

```
    int balanceFactor(AVLNode* node) {
        return height(node->left) - height(node->right);
    }
```

```
    void updateHeight(AVLNode* node) {
        node->height = 1 + max(height(node->left), height(node->right));
    }
```

```
    // Правый поворот
```

```
    AVLNode* rightRotate(AVLNode* y) {
        AVLNode* x = y->left;
        AVLNode* T2 = x->right;
```

```
        x->right = y;
        y->left = T2;
```

```
        updateHeight(y);
        updateHeight(x);
```

```
        return x;
    }
```

```
    // Левый поворот
```

```
    AVLNode* leftRotate(AVLNode* x) {
        AVLNode* y = x->right;
        AVLNode* T2 = y->left;
```

```

y->left = x;
x->right = T2;

updateHeight(x);
updateHeight(y);

return y;
}

AVLNode* balance(AVLNode* node) {
    if (node == nullptr) return nullptr;

    updateHeight(node);
    int bf = balanceFactor(node);

    // Левое поддерево выше
    if (bf > 1) {
        if (balanceFactor(node->left) < 0) {
            // Левый-правый поворот
            node->left = leftRotate(node->left);
        }
        // Правый поворот
        return rightRotate(node);
    }
    // Правое поддерево выше
    else if (bf < -1) {
        if (balanceFactor(node->right) > 0) {
            // Правый-левый поворот
            node->right = rightRotate(node->right);
        }
        // Левый поворот
        return leftRotate(node);
    }

    return node;
}

AVLNode* insert(AVLNode* node, int key) {
    if (node == nullptr) return new AVLNode(key);

    if (key < node->key) {
        node->left = insert(node->left, key);
    }

```

```

    } else if (key > node->key) {
        node->right = insert(node->right, key);
    } else {
        return node; // дубликаты не вставляем
    }

    return balance(node);
}

void inOrder(AVLNode* node) {
    if (node == nullptr) return;
    inOrder(node->left);
    cout << node->key << " ";
    inOrder(node->right);
}

void printTree(AVLNode* node, int level = 0) {
    if (node == nullptr) return;
    printTree(node->right, level + 1);
    for (int i = 0; i < level; i++) cout << "  ";
    cout << node->key << " (h=" << node->height << ", bf=" <<
balanceFactor(node) << ")" << endl;
    printTree(node->left, level + 1);
}

void destroy(AVLNode* node) {
    if (node == nullptr) return;
    destroy(node->left);
    destroy(node->right);
    delete node;
}

public:
    AVLTree() : root(nullptr) {}

    ~AVLTree() { destroy(root); }

    void insert(int key) { root = insert(root, key); }

    void print() {
        cout << "AVL-дерево:" << endl;
        printTree(root);
    }

```

```

        cout << endl;
    }

    void printInOrder() {
        inOrder(root);
        cout << endl;
    }
};

int main() {
    AVLTree tree;

    cout << "=== Вставка элементов в AVL-дерево ===" << endl;

    // Элементы, вызывающие балансировку
    vector<int> values = {10, 20, 30, 40, 50, 25};

    for (int val : values) {
        cout << "\nВставка " << val << ":" << endl;
        tree.insert(val);
        tree.print();
    }

    cout << "\nСимметричный обход (отсортированный порядок): ";
    tree.printInOrder();

    return 0;
}

```

Ожидаемый вывод (визуализация):

=== Вставка элементов в AVL-дерево ===

Вставка 10:

AVL-дерево:

10 (h=1, bf=0)

Вставка 20:

AVL-дерево:

20 (h=2, bf=-1)

10 (h=1, bf=0)

Вставка 30:

AVL-дерево:

20 (h=2, bf=0)

10 (h=1, bf=0)

30 (h=1, bf=0)

...

Пример 2. Префиксное дерево (Trie)

Задача: Реализовать Trie с операциями вставки, поиска и поиска по префиксу.

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

class Trie {
private:
    struct TrieNode {
        TrieNode* children[26];
        bool isEndOfWord;

        TrieNode() : isEndOfWord(false) {
            for (int i = 0; i < 26; i++) {
                children[i] = nullptr;
            }
        }
    };

    TrieNode* root;

    void destroy(TrieNode* node) {
        if (node == nullptr) return;
        for (int i = 0; i < 26; i++) {
            destroy(node->children[i]);
        }
        delete node;
    }

public:
```

```
Trie() : root(new TrieNode()) {}
```

```
~Trie() { destroy(root); }
```

```
void insert(const string& word) {  
    TrieNode* current = root;  
    for (char ch : word) {  
        int index = ch - 'a';  
        if (current->children[index] == nullptr) {  
            current->children[index] = new TrieNode();  
        }  
        current = current->children[index];  
    }  
    current->isEndOfWord = true;  
    cout << "Слово \"" << word << "\" добавлено" << endl;  
}
```

```
bool search(const string& word) {  
    TrieNode* current = root;  
    for (char ch : word) {  
        int index = ch - 'a';  
        if (current->children[index] == nullptr) return false;  
        current = current->children[index];  
    }  
    return current->isEndOfWord;  
}
```

```
bool startsWith(const string& prefix) {  
    TrieNode* current = root;  
    for (char ch : prefix) {  
        int index = ch - 'a';  
        if (current->children[index] == nullptr) return false;  
        current = current->children[index];  
    }  
    return true;  
}
```

```
void collectWords(TrieNode* node, string current, vector<string>& result)  
{  
    if (node->isEndOfWord) {  
        result.push_back(current);  
    }  
}
```

```

        for (int i = 0; i < 26; i++) {
            if (node->children[i] != nullptr) {
                collectWords(node->children[i], current + char('a' + i), result);
            }
        }
    }
}

```

```

vector<string> getAllWords() {
    vector<string> result;
    collectWords(root, "", result);
    return result;
}

```

```

vector<string> getWordsByPrefix(const string& prefix) {
    TrieNode* current = root;
    for (char ch : prefix) {
        int index = ch - 'a';
        if (current->children[index] == nullptr) return {};
        current = current->children[index];
    }
}

```

```

    vector<string> result;
    collectWords(current, prefix, result);
    return result;
}
};

```

```

int main() {
    Trie trie;

```

```

    cout << "=== Префиксное дерево (Trie) ===" << endl;

```

```

    trie insert("cat");
    trie insert("car");
    trie insert("dog");
    trie insert("cart");
    trie insert("catfish");

```

```

    cout << "\nПоиск слов:" << endl;
    cout << "search(\"cat\"): " << (trie.search("cat") ? "найдено" : "не
найдено") << endl;

```

```

        cout << "search(\"car\"): " << (trie.search("car") ? "найдено" : "не
найдено") << endl;
        cout << "search(\"cattle\"): " << (trie.search("cattle") ? "найдено" : "не
найдено") << endl;

        cout << "\nПоиск по префиксу:" << endl;
        cout << "startsWith(\"ca\"): " << (trie.startsWith("ca") ? "true" : "false")
<< endl;
        cout << "startsWith(\"do\"): " << (trie.startsWith("do") ? "true" : "false")
<< endl;
        cout << "startsWith(\"ba\"): " << (trie.startsWith("ba") ? "true" : "false")
<< endl;

        cout << "\nВсе слова в Trie:" << endl;
        vector<string> allWords = trie.getAllWords();
        for (const auto& word : allWords) {
            cout << " " << word << endl;
        }

        cout << "\nСлова с префиксом \"ca\":" << endl;
        vector<string> caWords = trie.getWordsByPrefix("ca");
        for (const auto& word : caWords) {
            cout << " " << word << endl;
        }

        return 0;
    }

```

Ожидаемый вывод:

=== Префиксное дерево (Trie) ===

Слово "cat" добавлено  
 Слово "car" добавлено  
 Слово "dog" добавлено  
 Слово "cart" добавлено  
 Слово "catfish" добавлено

Поиск слов:

search("cat"): найдено  
 search("car"): найдено  
 search("cattle"): не найдено

Поиск по префиксу:

startsWith("ca"): true  
startsWith("do"): true  
startsWith("ba"): false

Все слова в Trie:

car  
cart  
cat  
catfish  
dog

Слова с префиксом "ca":

car  
cart  
cat  
catfish

### Индивидуальные задания

Часть 1. AVL-дерево

Каждый вариант содержит задание реализовать AVL-дерево с указанными операциями:

| № вар. | Обязательные операции                  | Дополнительное задание       |
|--------|----------------------------------------|------------------------------|
| 1      | Вставка, балансировка                  | Найти минимальный элемент    |
| 2      | Вставка, балансировка                  | Найти максимальный элемент   |
| 3      | Вставка, балансировка                  | Поиск элемента               |
| 4      | Вставка, балансировка, поиск           | Удаление (лист)              |
| 5      | Вставка, балансировка, поиск           | Удаление (один потомок)      |
| 6      | Вставка, балансировка, поиск           | Удаление (два потомка)       |
| 7      | Вставка, балансировка, поиск, удаление | Найти k-й наименьший элемент |

| № вар. | Обязательные операции                  | Дополнительное задание                         |
|--------|----------------------------------------|------------------------------------------------|
| 8      | Вставка, балансировка, поиск, удаление | Найти наименьший общий предок (LCA)            |
| 9      | Вставка, балансировка, поиск, удаление | Подсчёт узлов в диапазоне                      |
| 10     | Вставка, балансировка, поиск, удаление | Визуализация дерева                            |
| 11     | Вставка, балансировка, поиск, удаление | Проверка сбалансированности                    |
| 12     | Вставка, балансировка, поиск, удаление | Построение AVL из массива                      |
| 13     | Вставка, балансировка, поиск, удаление | Объединение двух AVL-деревьев                  |
| 14     | Вставка, балансировка, поиск, удаление | Восстановление AVL из отсортированного массива |
| 15     | Вставка, балансировка, поиск, удаление | Сравнение производительности с BST             |

## Часть 2. Префиксное дерево (Trie)

Каждый вариант содержит задание реализовать Trie с указанными операциями:

| № вар. | Обязательные операции             | Дополнительное задание |
|--------|-----------------------------------|------------------------|
| 1      | Вставка, поиск                    | Поиск по префиксу      |
| 2      | Вставка, поиск                    | Удаление слова         |
| 3      | Вставка, поиск, поиск по префиксу | Получение всех слов    |

| № вар. | Обязательные операции             | Дополнительное задание                       |
|--------|-----------------------------------|----------------------------------------------|
| 4      | Вставка, поиск, поиск по префиксу | Автодополнение (топ-5)                       |
| 5      | Вставка, поиск, удаление          | Подсчёт количества слов                      |
| 6      | Вставка, поиск, поиск по префиксу | Поиск самого длинного префикса               |
| 7      | Вставка, поиск, поиск по префиксу | Проверка, есть ли слово с заданным префиксом |
| 8      | Вставка, поиск, удаление          | Импорт слов из файла                         |
| 9      | Вставка, поиск, поиск по префиксу | Экспорт слов в файл                          |
| 10     | Вставка, поиск, удаление          | Поддержка букв верхнего регистра             |
| 11     | Вставка, поиск, поиск по префиксу | Поддержка цифр (0-9)                         |
| 12     | Вставка, поиск, удаление          | Поиск слов по маске (с подстановкой '?')     |
| 13     | Вставка, поиск, поиск по префиксу | Сжатое префиксное дерево (Radix Tree)        |
| 14     | Вставка, поиск, удаление          | Частотный словарь (счётчик вхождений)        |
| 15     | Вставка, поиск, поиск по префиксу | Автодополнение с учётом популярности         |

### Часть 3. Сравнительный анализ

Задание для всех вариантов:

1. Реализовать AVL-дерево и обычное BST
2. Сравнить производительность вставки и поиска для  $N = 1000$ , 5000, 10000 элементов:

3. Случайные данные
4. Отсортированные данные (худший случай для BST)
5. Построить таблицу/график зависимости времени от N

### **Контрольные вопросы**

1. AVL-дерево
2. Что такое AVL-дерево? В чём его отличие от обычного BST?
3. Что такое фактор баланса? Как он вычисляется?
4. Какие существуют типы поворотов в AVL-дереве?
5. Когда применяется правый поворот? Левый поворот?
6. Когда применяется левый-правый поворот? Правый-левый?
7. Какова сложность операций в AVL-дереве?
8. Почему AVL-дерево называется самобалансирующимся?
9. Как часто происходит балансировка при вставке?
10. Чем AVL-дерево отличается от красно-чёрного дерева?
11. В каких случаях AVL-дерево эффективнее BST?
12. Префиксное дерево (Trie)
13. Что такое Trie (префиксное дерево)? Для решения каких задач оно используется?
14. Какова сложность поиска в Trie? От чего она зависит?
15. Какая структура данных эффективнее для строк: Trie или хеш-таблица?
16. Как реализовать автодополнение с помощью Trie?
17. Каковы недостатки Trie? Какой объём памяти он требует?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (AVL-дерево, повороты, Trie)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода, визуализация)

Анализ сложности операций

Сравнительная таблица BST vs AVL vs Trie

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

| №     | Критерий                             | Баллы |
|-------|--------------------------------------|-------|
| 1     | AVL-дерево                           | 45    |
| 1.1   | Реализация узла и высот              | 5     |
| 1.2   | Реализация поворотов                 | 15    |
| 1.3   | Реализация балансировки              | 10    |
| 1.4   | Реализация вставки                   | 10    |
| 1.5   | Реализация удаления (по варианту)    | 5     |
| 2     | Префиксное дерево (Trie)             | 35    |
| 2.1   | Структура узла Trie                  | 5     |
| 2.2   | Реализация вставки                   | 10    |
| 2.3   | Реализация поиска                    | 10    |
| 2.4   | Реализация дополнительных операций   | 10    |
| 3     | Сравнительный анализ                 | 10    |
| 3.1   | Измерение времени выполнения         | 5     |
| 3.2   | Таблица/график сравнения             | 5     |
| 4     | Оформление и отчёт                   | 10    |
| 4.1   | Код отформатирован, есть комментарии | 5     |
| 4.2   | Ссылка на Git-репозиторий            | 5     |
| Итого |                                      | 100   |

Штрафные баллы

| Нарушение                                 | Штраф                 |
|-------------------------------------------|-----------------------|
| Некорректная балансировка (нет поворотов) | -20                   |
| Утечка памяти (не освобождены узлы)       | -15                   |
| Неверная реализация поиска по префиксу    | -10                   |
| Нет сравнительного анализа                | -10                   |
| Нет ссылки на Git-репозиторий             | -10                   |
| Программа не компилируется                | работа не принимается |

#### Итоговая шкала

| Оценка              | Баллы    |
|---------------------|----------|
| Отлично             | 90–100   |
| Хорошо              | 75–89    |
| Удовлетворительно   | 60–74    |
| Неудовлетворительно | менее 60 |